

02_PCA

Anish

2026-04-25

1. Why PCA before clustering

Two distinct jobs PCA does for this project, both straight

1. **Visualisation** - project the the customers onto PC1-PC2 so we can eyeball whether segments separates in the plane of the greatest variance, colour by candidate cluster labels later, and produce the biplot.
2. **Preprocessing** - replace the seven correlated input features with 2-3 orthogonal PC scores and cluster those.

The output of this notebook is stored in PC-scores matrix ('../data/customer_pc_scores.csv') consumed by '03_clustering.Rmd'

2. Load the customer-level features

```
customer <- read_csv("../data/customer_features.csv",  
                      show_col_types = FALSE)  
glimpse(customer)
```

```
## Rows: 4,334  
## Columns: 21  
## $ CustomerID      <dbl> 12346, 12347, 12348, 12349, 12350, 12352, 12353, 12...  
## $ Recency          <dbl> 326, 2, 75, 19, 310, 36, 204, 232, 214, 23, 33, 2, ...  
## $ Frequency        <dbl> 1, 7, 4, 1, 1, 7, 1, 1, 1, 3, 1, 2, 4, 3, 1, 10, 2,...  
## $ Monetary        <dbl> 77183.60, 4310.00, 1437.24, 1457.55, 294.40, 1385.7...  
## $ TotalQuantity    <dbl> 74215, 2458, 2332, 630, 196, 526, 20, 530, 240, 157...  
## $ UniqueProducts   <dbl> 1, 103, 21, 72, 16, 57, 4, 58, 13, 52, 131, 12, 214...  
## $ AvgUnitPrice      <dbl> 1.040000, 2.644011, 0.692963, 4.237500, 1.581250, 4...  
## $ FirstPurchase    <dtm> 2011-01-18 10:01:00, 2010-12-07 14:57:00, 2010-12-...  
## $ LastPurchase     <dtm> 2011-01-18 10:01:00, 2011-12-07 15:52:00, 2011-09-...  
## $ TenureDays        <dbl> 0, 365, 282, 0, 0, 260, 0, 0, 0, 302, 0, 149, 274, ...  
## $ AvgOrderValue     <dbl> 77183.6000, 615.7143, 359.3100, 1457.5500, 294.4000...  
## $ AvgOrderItems     <dbl> 74215.00000, 351.14286, 583.00000, 630.00000, 196.0...  
## $ ReturnInvoices    <dbl> 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 2, 0, 0, 3, 0, ...  
## $ Country           <chr> "United Kingdom", "Iceland", "Finland", "Italy", "N...  
## $ ReturnRate        <dbl> 0.5000000, 0.0000000, 0.0000000, 0.0000000, 0.00000...  
## $ log_Recency       <dbl> 5.789960, 1.098612, 4.330733, 2.995732, 5.739793, 3...  
## $ log_Frequency     <dbl> 0.6931472, 2.0794415, 1.6094379, 0.6931472, 0.69314...  
## $ log_Monetary      <dbl> 11.253955, 8.368925, 7.271175, 7.285198, 5.688330, ...  
## $ log_TotalQuantity <dbl> 11.214735, 7.807510, 7.754910, 6.447306, 5.283204, ...  
## $ log_UniqueProducts <dbl> 0.6931472, 4.6443909, 3.0910425, 4.2904594, 2.83321...  
## $ log_AvgOrderValue <dbl> 11.253955, 6.424406, 5.886965, 7.285198, 5.688330, ...
```

3. Build the modelling matrix

From the EDA (correlation heatmap in 01_EDA.Rmd) we settled on a seven-feature input. The redundant ones (AvgBasketValue $\approx$ AvgOrderValue , TotalQuantity $\approx$ Monetary , ReturnInvoices $\approx$ Frequency) were dropped. Country was dropped (82.9% UK). ReturnRate is held back as a *profiling* variable — too sparse to anchor a cluster, but useful for interpretation later.

```
feature_cols <- c("log_Recency", "log_Frequency", "log_Monetary",  
                  "log_UniqueProducts", "log_AvgOrderValue",  
                  "AvgUnitPrice", "TenureDays")
```

```
X <- customer |>  
  select(CustomerID, all_of(feature_cols)) |>  
  drop_na()
```

```
dim(X)
```

```
## [1] 4334    8
```

```
summary(X[, feature_cols])
```

```
##   log_Recency    log_Frequency    log_Monetary    log_UniqueProducts  
##   Min.      :0.6931    Min.      :0.6931    Min.      : 1.558    Min.      :0.6931  
##   1st Qu.:2.9444    1st Qu.:0.6931    1st Qu.: 5.721    1st Qu.:2.8332  
##   Median :3.9512    Median :1.0986    Median : 6.498    Median :3.5835  
##   Mean   :3.8321    Mean   :1.3416    Mean   : 6.575    Mean   :3.5496  
##   3rd Qu.:4.9698    3rd Qu.:1.7918    3rd Qu.: 7.398    3rd Qu.:4.3567  
##   Max.    :5.9269    Max.    :5.3327    Max.    :12.539    Max.    :7.4877  
##   log_AvgOrderValue AvgUnitPrice      TenureDays  
##   Min.      : 1.558    Min.      : 0.1225    Min.      : 0.0  
##   1st Qu.: 5.183    1st Qu.: 2.1548    1st Qu.: 0.0  
##   Median : 5.672    Median : 2.8275    Median : 92.0  
##   Mean   : 5.638    Mean   : 3.1197    Mean   :130.3  
##   3rd Qu.: 6.051    3rd Qu.: 3.6892    3rd Qu.:252.0  
##   Max.    :11.341    Max.    :10.6743    Max.    :373.0
```

4. Fit PCA

prcomp with scale = TRUE standardises every column to zero mean and unit variance — required because AvgUnitPrice and TenureDays are on totally different scales from the log features.

```
pr.out <- prcomp(X[, feature_cols], center = TRUE, scale. = TRUE)  
pr.out
```

```
## Standard deviations (1, .., p=7):
## [1] 1.92727152 1.06680187 1.00673007 0.72909391 0.62645596 0.45542400 0.05115491
##
## Rotation (n x k) = (7 x 7):
##
##           PC1          PC2          PC3          PC4          PC5
## log_Recency    -0.35246166 -0.34795910 -0.02892215 -0.86516438  0.07073114
## log_Frequency   0.45908761  0.24809202 -0.15467220 -0.25286175  0.19148878
## log_Monetary    0.48286695 -0.28381251 -0.05615320 -0.05604786  0.24007082
## log_UniqueProducts 0.41896501 -0.09162794  0.14407614 -0.21071672 -0.86386868
## log_AvgOrderValue 0.25548223 -0.78553480  0.09114461  0.22055280  0.18776887
## AvgUnitPrice   -0.07571219 -0.15070199 -0.95806074  0.10634642 -0.20512124
## TenureDays      0.43048074  0.29774753 -0.15855970 -0.28294085  0.27765839
##
##           PC6          PC7
## log_Recency    -0.01860633 -0.0005044326
## log_Frequency  -0.59663611 -0.4971757823
## log_Monetary   -0.25360175  0.7470258183
## log_UniqueProducts 0.06739411 -0.0101047016
## log_AvgOrderValue 0.17943465 -0.4396124347
## AvgUnitPrice   0.01707338 -0.0006377597
## TenureDays     0.73643401 -0.0375074575
```

5. Variance explained

```
pve <- pr.out$sdev^2 / sum(pr.out$sdev^2)
pve_df <- tibble(
  PC          = factor(paste0("PC", seq_along(pve)),
                        levels = paste0("PC", seq_along(pve))),
  PVE          = pve,
  cumulative   = cumsum(pve)
)
pve_df
```

PC <fct>	PVE <dbl>	cumulative <dbl>
PC1	0.530625071	0.5306251
PC2	0.162580889	0.6932060
PC3	0.144786490	0.8379925
PC4	0.075939705	0.9139322
PC5	0.056063867	0.9699960
PC6	0.029630146	0.9996262
PC7	0.000373832	1.0000000

7 rows

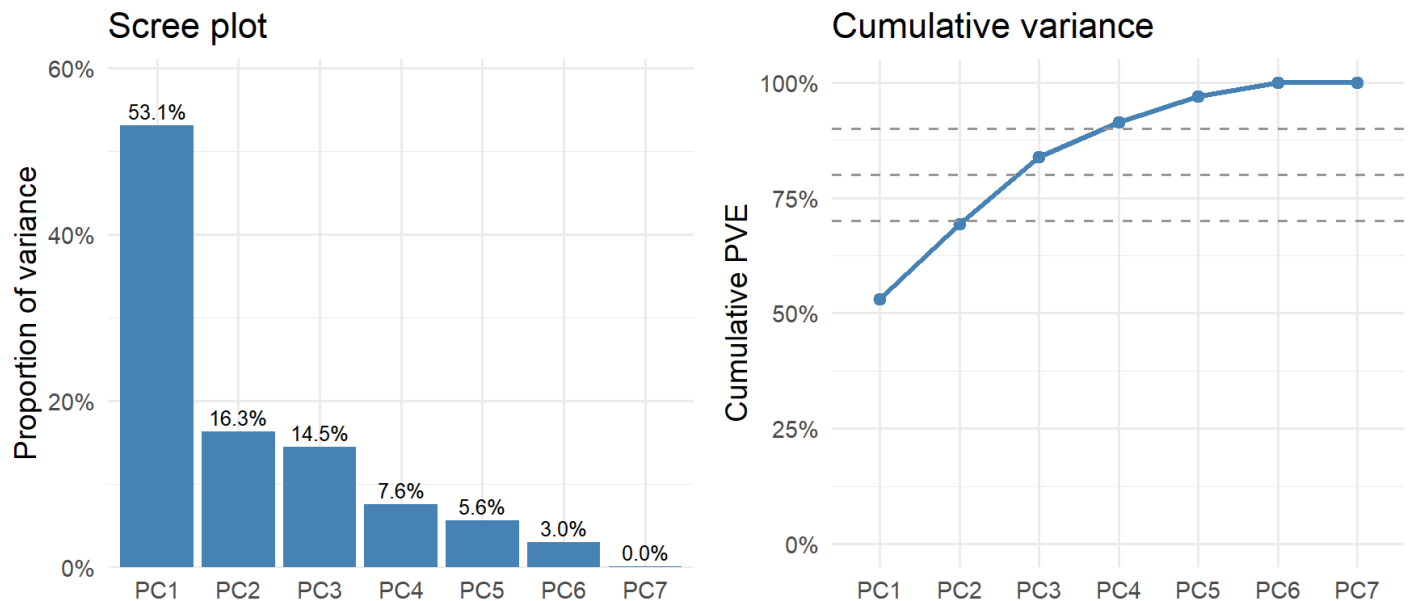
```

p_screes <- ggplot(pve_df, aes(PC, PVE)) +
  geom_col(fill = "steelblue") +
  geom_text(aes(label = percent(PVE, accuracy = 0.1)),
    vjust = -0.4, size = 3) +
  scale_y_continuous(labels = percent_format(),
    expand = expansion(mult = c(0, 0.15))) +
  labs(title = "Scree plot", x = NULL, y = "Proportion of variance")

p_cum <- ggplot(pve_df, aes(PC, cumulative, group = 1)) +
  geom_line(colour = "steelblue", linewidth = 1) +
  geom_point(colour = "steelblue", size = 2) +
  geom_hline(yintercept = c(0.7, 0.8, 0.9),
    linetype = "dashed", colour = "grey60") +
  scale_y_continuous(labels = percent_format(), limits = c(0, 1)) +
  labs(title = "Cumulative variance",
    x = NULL, y = "Cumulative PVE")

gridExtra::grid.arrange(p_screes, p_cum, nrow = 1)

```



```

tibble(
  PC = paste0("PC", seq_along(pr.out$sdev)),
  eigenvalue = pr.out$sdev^2,
  keep_kaiser = pr.out$sdev^2 > 1
)

```

PC <chr>	eigenvalue <dbl>	keep_kaiser <lgl>
PC1	3.714375498	TRUE
PC2	1.138066225	TRUE
PC3	1.013505432	TRUE

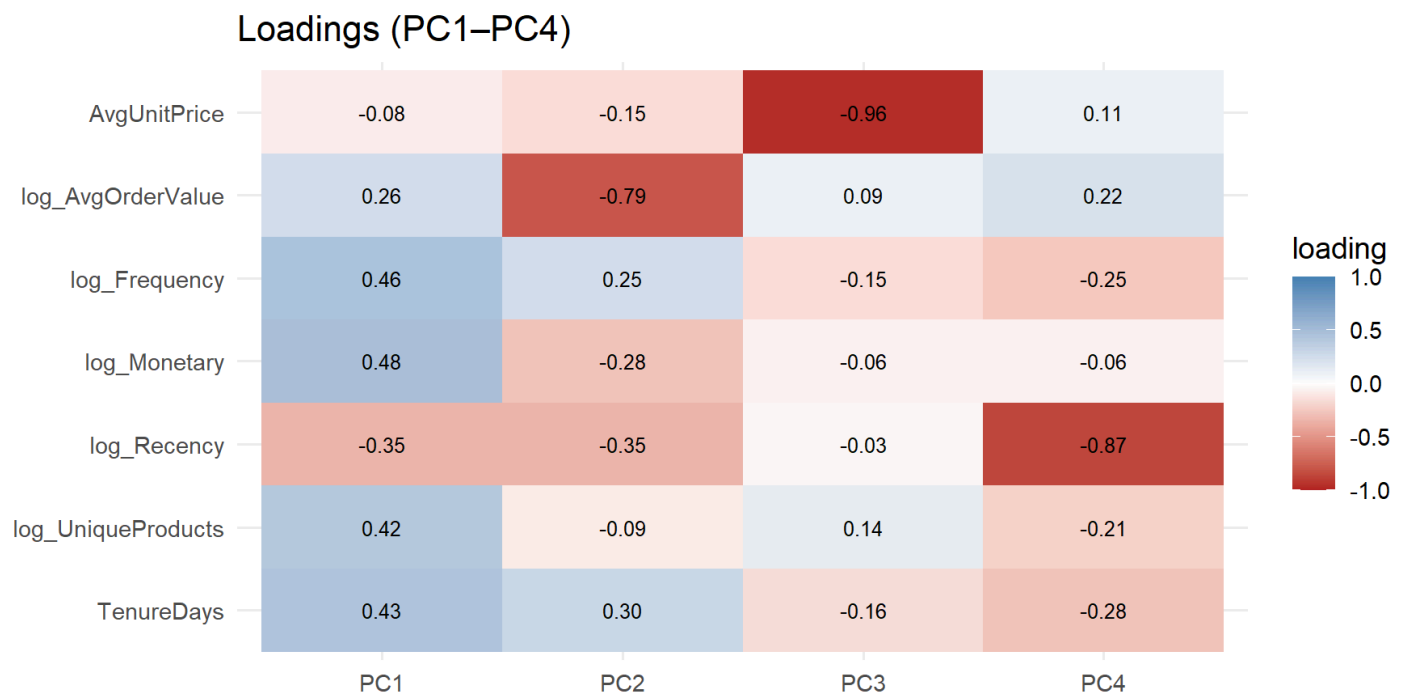
PC <chr>	eigenvalue <dbl>	keep_kaiser <lgl>
PC4	0.531577934	FALSE
PC5	0.392447068	FALSE
PC6	0.207411019	FALSE
PC7	0.002616824	FALSE
7 rows		

6. Loadings — what each PC actually means

```
loadings <- as_tibble(pr.out$rotation, rownames = "feature")
loadings |> mutate(across(where(is.numeric), \(x) round(x, 2)))
```

feature <chr>	PC1 <dbl>	PC2 <dbl>	PC3 <dbl>	PC4 <dbl>	PC5 <dbl>	PC6 <dbl>	PC7 <dbl>
log_Recency	-0.35	-0.35	-0.03	-0.87	0.07	-0.02	0.00
log_Frequency	0.46	0.25	-0.15	-0.25	0.19	-0.60	-0.50
log_Monetary	0.48	-0.28	-0.06	-0.06	0.24	-0.25	0.75
log_UniqueProducts	0.42	-0.09	0.14	-0.21	-0.86	0.07	-0.01
log_AvgOrderValue	0.26	-0.79	0.09	0.22	0.19	0.18	-0.44
AvgUnitPrice	-0.08	-0.15	-0.96	0.11	-0.21	0.02	0.00
TenureDays	0.43	0.30	-0.16	-0.28	0.28	0.74	-0.04
7 rows							

```
loadings |>
  pivot_longer(-feature, names_to = "PC", values_to = "loading") |>
  filter(PC %in% paste0("PC", 1:4)) |>
  mutate(PC = factor(PC, levels = paste0("PC", 1:4))) |>
  ggplot(aes(PC, fct_rev(feature), fill = loading)) +
  geom_tile() +
  geom_text(aes(label = sprintf("%.2f", loading)), size = 3) +
  scale_fill_gradient2(low = "firebrick", mid = "white",
    high = "steelblue", midpoint = 0,
    limits = c(-1, 1)) +
  labs(title = "Loadings (PC1-PC4)", x = NULL, y = NULL)
```



Reading the loadings is what lets us *name* each component. Expected pattern based on the EDA correlation structure:

- **PC1** loads positively on `log_Frequency` , `log_Monetary` , `log_UniqueProducts` , `log_AvgOrderValue` and negatively on `log_Recency` — i.e. an “engagement / customer value” axis.
- **PC2** is dominated by `TenureDays` and either `AvgUnitPrice` or `Recency` ’s residual signal — a “tenure / lifecycle” axis orthogonal to engagement.
- **PC3** isolates `AvgUnitPrice` (which the EDA showed was near-uncorrelated with everything else) — a “price-point” axis.

7. PC scores — the new feature space

```
scores <- as_tibble(pr.out$x) |>
  mutate(CustomerID = X$CustomerID, .before = 1) |>
  left_join(customer |> select(CustomerID, Monetary, Frequency,
                              Recency, Country, ReturnRate),
            by = "CustomerID")

head(scores)
```

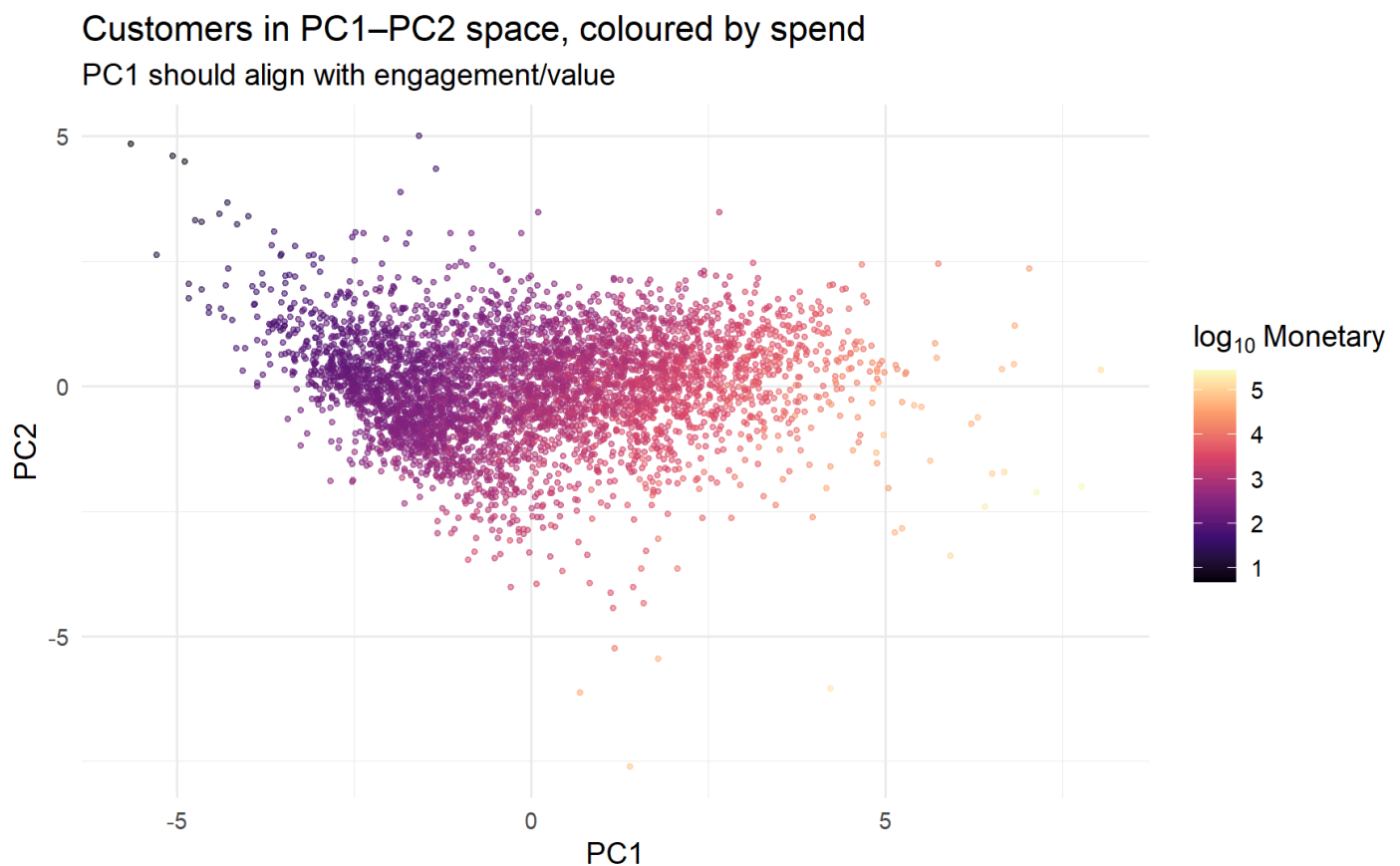
CustomerID <dbl>	PC1 <dbl>	PC2 <dbl>	PC3 <dbl>	PC4 <dbl>	PC5 <dbl>	PC6 <dbl>	PC7 <dbl>
12346	1.3980174	-7.5967590	1.65612072	1.1027269	4.413507984	0.03108592	-0.0000000
12347	3.3686184	0.2283065	0.05784925	0.9043135	0.325450246	0.58811354	-0.0000000
12348	0.8450484	0.1619330	1.17999522	-0.7837433	1.285703894	0.47009744	0.0000000
12349	0.4173196	-2.3812020	-0.10153858	1.4545589	-0.660812095	0.16312041	-0.0000000
12350	-1.8772277	-0.6700957	1.16329866	-0.6286942	0.236678281	-0.05352601	-0.0000000

CustomerID <dbl>	PC1 <dbl>	PC2 <dbl>	PC3 <dbl>	PC4 <dbl>	PC5 <dbl>	PC6 <dbl>	
12352	1.2552178	0.7001810	-0.91296856	-0.5704532	-0.008425634	-0.09506090	0.0

6 rows | 14 of 12 columns

7a. PC1 vs PC2 scatter, coloured by spend

```
ggplot(scores, aes(PC1, PC2, colour = log10(Monetary + 1))) +
  geom_point(alpha = 0.5, size = 0.8) +
  scale_colour_viridis_c(option = "magma",
                        name = expression(log[10]~Monetary)) +
  labs(title = "Customers in PC1-PC2 space, coloured by spend",
       subtitle = "PC1 should align with engagement/value")
```

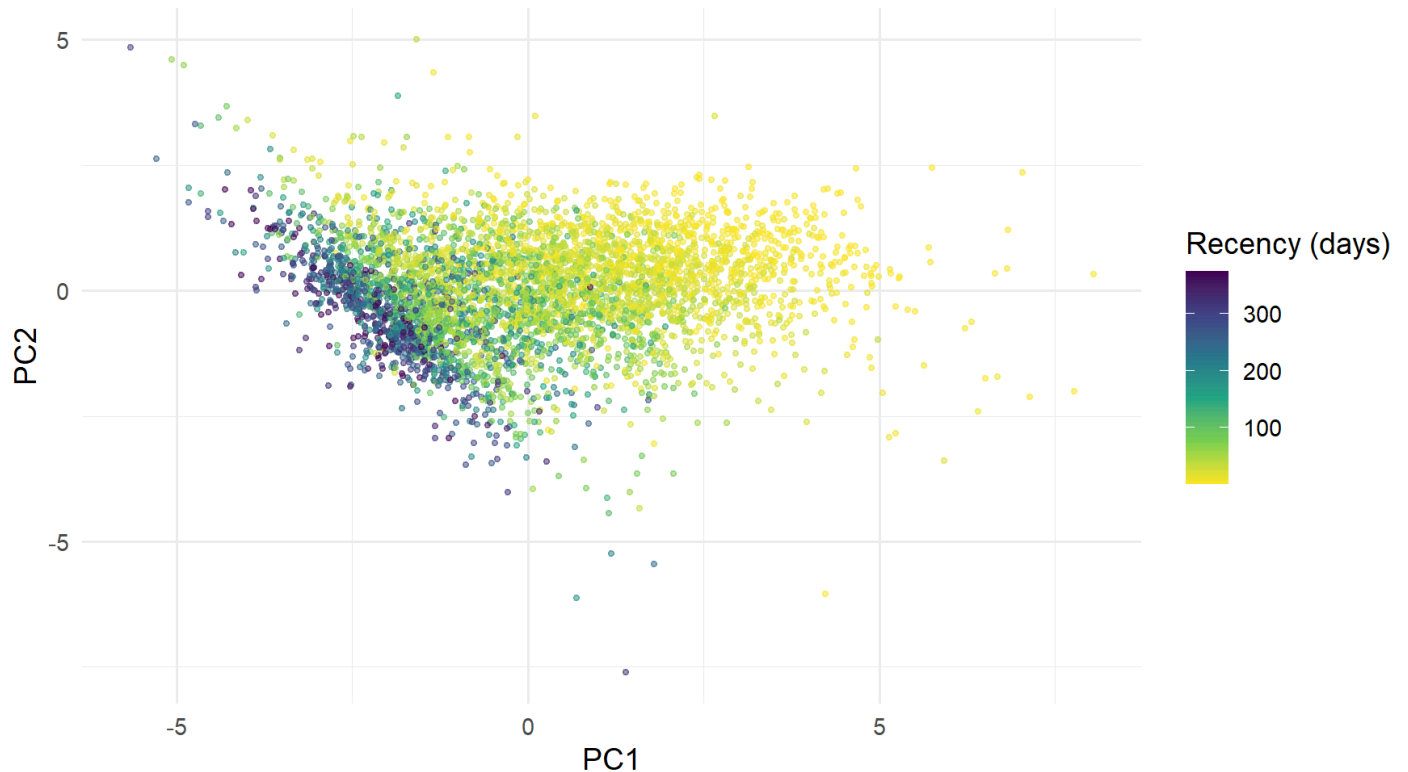


7b. PC1 vs PC2 scatter, coloured by recency

```
ggplot(scores, aes(PC1, PC2, colour = Recency)) +
  geom_point(alpha = 0.5, size = 0.8) +
  scale_colour_viridis_c(option = "viridis", direction = -1,
                        name = "Recency (days)") +
  labs(title = "PC1-PC2 space, coloured by recency",
       subtitle = "Recent customers should sit on the engagement side of PC1")
```

PC1–PC2 space, coloured by recency

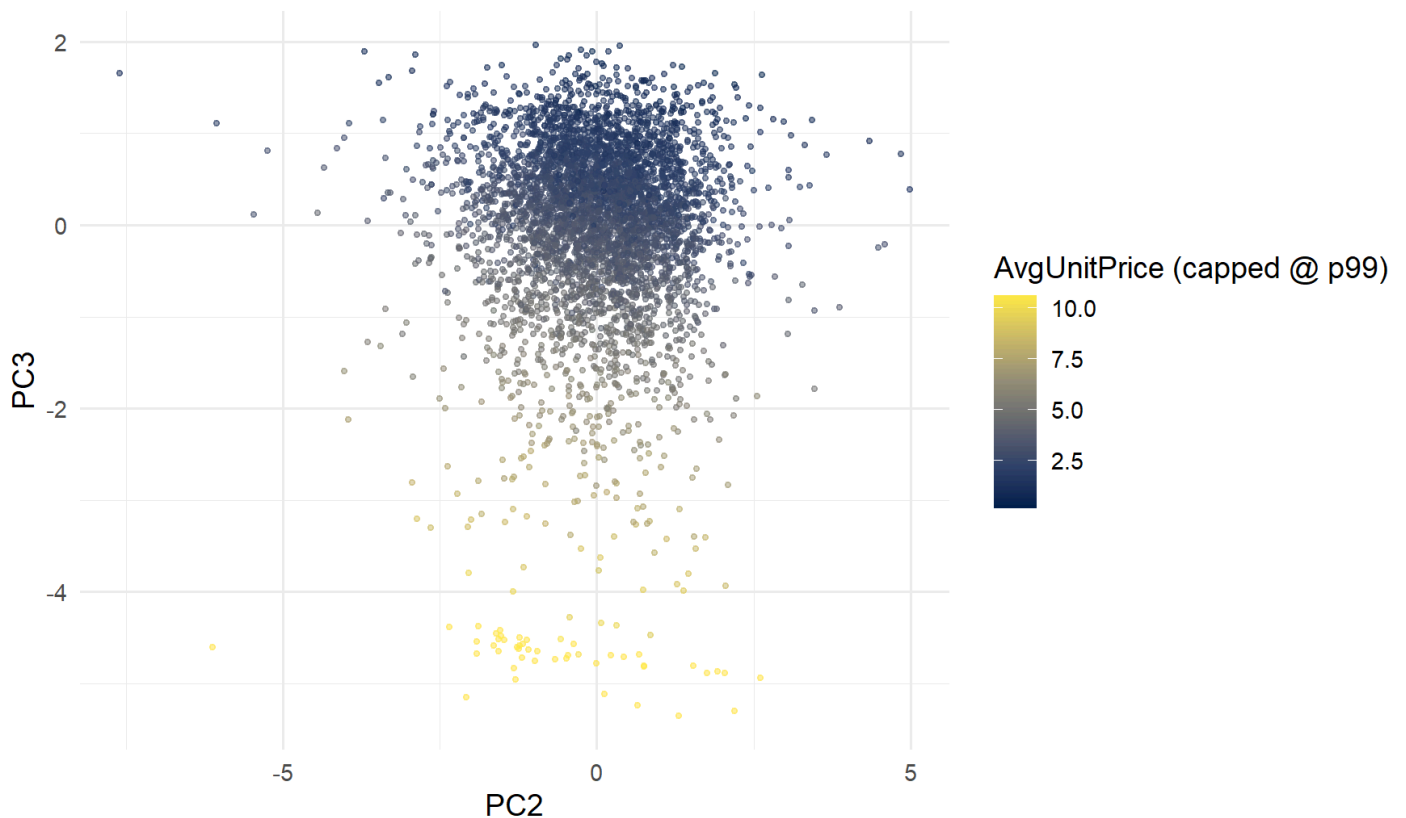
Recent customers should sit on the engagement side of PC1



7c. PC2 vs PC3, coloured by AvgUnitPrice

```
ggplot(scores, aes(PC2, PC3,  
  colour = pmin(customer$AvgUnitPrice[match(CustomerID,  
                                                customer$CustomerID)],  
                quantile(customer$AvgUnitPrice, 0.99,  
                          na.rm = TRUE)))) +  
geom_point(alpha = 0.5, size = 0.8) +  
scale_colour_viridis_c(option = "cividis",  
  name = "AvgUnitPrice (capped @ p99)") +  
labs(title = "PC2 vs PC3 – surfacing the price-point axis")
```

PC2 vs PC3 — surfacing the price-point axis



8. Biplot

That works, but a ggplot version reads more clearly in the report. We overlay variable arrows (scaled to the score range) on top of the scatter.

```
arrow_scale <- with(pr.out,
  min(max(abs(x[, 1])) / max(abs(rotation[, 1])),
    max(abs(x[, 2])) / max(abs(rotation[, 2])))) * 0.7
```

```
arrows_df <- as_tibble(pr.out$rotation, rownames = "feature") |>
  mutate(PC1 = PC1 * arrow_scale,
    PC2 = PC2 * arrow_scale)
```

```
ggplot(scores, aes(PC1, PC2)) +
  geom_point(alpha = 0.25, size = 0.6, colour = "grey50") +
  geom_segment(data = arrows_df,
    aes(x = 0, y = 0, xend = PC1, yend = PC2),
    arrow = arrow(length = unit(0.2, "cm")),
    colour = "firebrick", linewidth = 0.7) +
  geom_text(data = arrows_df,
    aes(x = PC1 * 1.08, y = PC2 * 1.08, label = feature),
    colour = "firebrick", size = 3.5) +
  labs(title = "Biplot – customers + variable loadings on PC1-PC2",
    x = sprintf("PC1 (%s)", percent(pve[1], accuracy = 0.1)),
    y = sprintf("PC2 (%s)", percent(pve[2], accuracy = 0.1)))
```

Biplot — customers + variable loadings on PC1–PC2



9. Outlier check on PC1

The EDA Pareto curve suggested the top ~1% of customers might pull a disproportionate amount of variance. With log scaling that should be much tamer, but worth confirming on PC1.

```
scores |>
  arrange(desc(PC1)) |>
  slice_head(n = 10) |>
  select(CustomerID, PC1, PC2, Monetary, Frequency, Recency, Country)
```

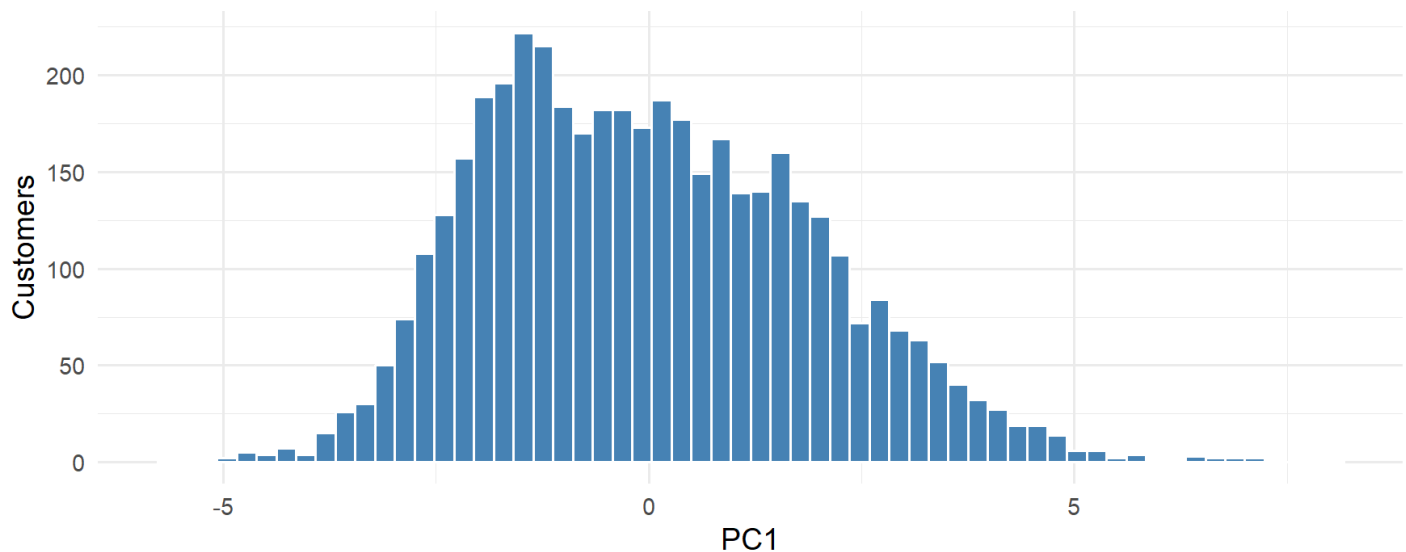
CustomerID <dbl>	PC1 <dbl>	PC2 <dbl>	Monetary <dbl>	Frequency <dbl>	Recency <dbl>	Country <chr>
14911	8.047071	0.3248214	136161.83	198	1	EIRE
14646	7.772339	-2.0101672	279138.02	72	2	Netherlands
18102	7.140339	-2.1206914	259657.30	60	1	United Kingdom

CustomerID <dbl>	PC1 <dbl>	PC2 <dbl>	Monetary <dbl>	Frequency <dbl>	Recency <dbl>	Country <chr>
12748	7.041259	2.3402199	31650.78	206	1	United Kingdom
17841	6.829754	1.2084907	40495.99	124	2	United Kingdom
15311	6.824751	0.4347397	60632.75	91	1	United Kingdom
14156	6.687506	-1.7151842	116560.08	54	10	EIRE
13089	6.648899	0.3382603	58762.08	97	3	United Kingdom
17511	6.514556	-1.7566448	91062.38	31	3	United Kingdom
17450	6.411752	-2.4070189	194390.79	46	8	United Kingdom

1-10 of 10 rows

```
ggplot(scores, aes(PC1)) +
  geom_histogram(bins = 60, fill = "steelblue", colour = "white") +
  labs(title = "Distribution of PC1 (engagement axis)",
       x = "PC1", y = "Customers")
```

Distribution of PC1 (engagement axis)



PC1 looks roughly symmetric and unimodal so we're fine to cluster on it directly.

10. Save PC scores for the clustering step

We keep the first k PCs where k is the smallest number that crosses 80% cumulative variance — defensible under both the elbow heuristic and the variance-threshold heuristic. Adjust k here if the scree suggests otherwise.

```
k <- which(cumsum(pve) >= 0.80)[1]
k
```

```
## [1] 3
```

```
scores_out <- scores |>
  select(CustomerID, all_of(paste0("PC", 1:k)))

write_csv(scores_out, "../data/customer_pc_scores.csv")

list(
  components_kept = k,
  variance_kept   = percent(sum(pve[1:k]), accuracy = 0.1),
  customers       = nrow(scores_out)
)
```

```
## $components_kept
## [1] 3
##
## $variance_kept
## [1] "83.8%"
##
## $customers
## [1] 4334
```